

Proyecto 1: Consumo de API JSON con AJAX y Bootstrap

Objetivo del proyecto para el alumno

- Aprender a hacer **peticiones AJAX** al servidor y procesar la respuesta en JSON.
- Mostrar los datos en la página de forma dinámica usando **HTML y Bootstrap**.
- Comprender la comunicación **cliente** → **servidor** → **cliente**.

Lo que se espera que haga el alumno:

1. Iniciar un servidor Node.js con Express que devuelva datos de productos en JSON.
2. Crear una página web con Bootstrap que muestre los productos en una **tabla dinámica**.
3. Usar **Fetch API** para solicitar los datos al servidor y mostrarlos sin recargar la página.
4. Realizar ejercicios de modificación, filtrado y ordenamiento de productos.

1. Código del servidor con Express (Node.js)

1.1 Crear el proyecto

```
# Crear carpeta del proyecto
mkdir proyecto_ajax
cd proyecto_ajax

# Inicializar proyecto Node.js
npm init -y

# Instalar Express
npm install express
```

1.2 Crear el servidor (**server.js**)

```
const express = require('express');
const app = express();

// Permitir solicitudes CORS desde el cliente (si el cliente está en
otro puerto)
const cors = require('cors');
app.use(cors());

// Datos simulados (JSON)
const productos = [
  { id: 1, nombre: "Laptop", categoria: "Electrónica", precio: 1200 },
  { id: 2, nombre: "Mouse", categoria: "Electrónica", precio: 25 },
];
```

```

    { id: 3, nombre: "Teclado", categoria: "Electrónica", precio: 45
  },
  { id: 4, nombre: "Leche", categoria: "Alimentos", precio: 2 },
  { id: 5, nombre: "Pan", categoria: "Alimentos", precio: 1.5 }
];

// Ruta principal: devuelve lista de productos en JSON
app.get('/api/productos', (req, res) => {
  res.json(productos);
});

// Ruta con parámetro opcional: filtrar por categoría
app.get('/api/productos/:categoria', (req, res) => {
  const categoria = req.params.categoria.toLowerCase();
  const filtrados = productos.filter(p => p.categoria.toLowerCase()
=== categoria);
  res.json(filtrados);
});

// Iniciar servidor
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
});

```

1.3 Ejecutar el servidor

```

# Ejecutar servidor
node server.js

```

- **Puerto: 3000**
- **Peticiones posibles:**
 - GET `http://localhost:3000/api/productos` → devuelve todos los productos
 - GET `http://localhost:3000/api/productos/Electrónica` → devuelve solo productos de la categoría Electrónica

Funcionamiento:

- El servidor está listo y devuelve datos en **JSON**.
- Ahora el cliente puede hacer peticiones **AJAX** para mostrarlos.

2. Código del cliente HTML + Bootstrap + AJAX

2.1 HTML con Bootstrap

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Productos AJAX</title>
  <!-- Bootstrap CSS CDN -->

```

```

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

    <h1 class="mb-4">Lista de Productos</h1>

    <!-- Filtro de categoría -->
    <div class="mb-3">
        <input type="text" id="categoria" class="form-control"
placeholder="Filtrar por categoría">
        <button id="btnFiltrar" class="btn btn-primary mt-
2">Filtrar</button>
        <button id="btnCargarTodos" class="btn btn-secondary mt-
2">Cargar Todos</button>
    </div>

    <!-- Tabla para mostrar productos -->
    <table class="table table-striped">
        <thead>
            <tr>
                <th>ID</th>
                <th>Nombre</th>
                <th>Categoría</th>
                <th>Precio</th>
            </tr>
        </thead>
        <tbody id="tablaProductos">
            <!-- Productos se cargarán aquí dinámicamente -->
        </tbody>
    </table>

    <script src="script.js"></script>
</body>
</html>

```

2.2 JavaScript (`script.js`) usando Fetch

```

const tabla = document.getElementById('tablaProductos');
const btnCargarTodos = document.getElementById('btnCargarTodos');
const btnFiltrar = document.getElementById('btnFiltrar');
const inputCategoria = document.getElementById('categoria');

const API_BASE = 'http://localhost:3000/api/productos';

// Función para mostrar productos en la tabla
function mostrarProductos(productos) {
    tabla.innerHTML = '';
    productos.forEach(p => {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${p.id}</td>
            <td>${p.nombre}</td>
            <td>${p.categoria}</td>
            <td>${p.precio}</td>
        `;
        tabla.appendChild(row);
    });
}

```

```

}

// Cargar todos los productos
btnCargarTodos.addEventListener('click', () => {
  fetch(API_BASE)
    .then(res => res.json())
    .then(data => mostrarProductos(data))
    .catch(err => console.error(err));
});

// Filtrar por categoría
btnFiltrar.addEventListener('click', () => {
  const categoria = inputCategoria.value.trim();
  if (categoria === '') {
    alert('Introduce una categoría');
    return;
  }
  fetch(`${API_BASE}/${encodeURIComponent(categoria)}`)
    .then(res => res.json())
    .then(data => mostrarProductos(data))
    .catch(err => console.error(err));
});

```

2.3 Funcionamiento del proyecto

1. El alumno abre `index.html` en el navegador.
2. Hace click en **Cargar Todos** → la tabla se llena con todos los productos desde el servidor.
3. Escribe una categoría en el input y hace click en **Filtrar** → se muestran solo los productos de esa categoría.
4. Todo sucede **sin recargar la página**, gracias a AJAX.

3. Ejercicios propuestos para los alumnos

1. **Agregar precio total:** mostrar debajo de la tabla la suma de los precios de los productos cargados.
2. **Ordenar por precio:** agregar botones para ordenar ascendente o descendente por precio.
3. **Agregar nuevos productos:** crear un formulario para añadir productos al servidor (POST) y actualizar la tabla automáticamente.
4. **Búsqueda avanzada:** permitir filtrar por categoría y un precio máximo a la vez.
5. **Diseño con Bootstrap:** hacer que la tabla sea responsive y resalte productos de más de \$100 con un color diferente.